ICG WITH BACKPATCHING

One-Pass Intermediate Code Generation for Control-Flow Statements using Flex and Bison

Project Report

BCSE307P Compiler Design Lab

Team 15

Prithvi Raghu (24BCE2624)

Jai Narayan Jayaprakash Jayanthi (24BCE2751)

Divyanshu Pal (24BCE2856)

Faculty: Dr. Naga Priyadarsini

October 2026

Abstract

A compiler that emits code while it parses meets jumps whose targets do not exist yet: when the condition of an `if` is read, the code that follows the then-branch has not been generated. This project implements backpatching, the textbook solution in which each such jump is emitted with an empty target, recorded in a list, and filled the moment its target becomes known. The generator translates a small C-like language with assignments, arithmetic, short-circuit boolean conditions, `if`, `if-else`, `while`, `do-while`, `for`, `break` and `continue` into numbered three-address code in a single pass, and is written entirely as a Flex lexer and a Bison grammar with C semantic actions.

Two additions go beyond the textbook scheme. First, a trace mode prints every `emit`, `makelist`, `merge` and `backpatch` operation as it happens, with an optional step-by-step mode for live walkthroughs, so that the normally invisible patch lists can be watched while code is generated. Second, a two-pass baseline written as a second Bison grammar generates the same code the conventional way, through a syntax tree and symbolic labels. Both programs produce identical quadruples for all 15 code-generating test programs, which cross-checks every semantic action, and the comparison quantifies what each approach must hold in memory until jump targets are known.

Keywords: intermediate code generation, backpatching, three-address code, short-circuit evaluation, Flex, Bison.

1. Introduction

1.1 The problem: forward jumps in one pass

Intermediate code generation turns a parsed program into a simpler, machine-independent form such as three-address code. A syntax-directed translator built on an LR parser generates this code inside the semantic actions, as each grammar rule is reduced. This is efficient, because no syntax tree is stored, but it creates a difficulty for control flow. Consider the program below.

```
x = a + b;
if (a < b) y = c;

100: t1 = a + b
101: x = t1
102: if a < b goto _
103: goto _         <- where?
104: y = c
105: halt
```

When the condition `a < b` is reduced, the generator must emit the jump taken when the condition is false (quad 103). That jump should land just after the then-branch, at quad 105, but quad 105 has not been generated yet. Either the targets are resolved later, with symbolic labels and a second pass, or the empty jump is remembered and filled once the target is known. The second technique is backpatching (Aho et al., sec. 6.7), and it is the subject of this project.

1.2 Objectives

1. Correct three-address code for assignment, arithmetic, `if`, `if-else`, `while`, `do-while`, `for`, `break` and `continue`, nested to any depth, as numbered quadruples with no jump left unfilled.
2. A single pass: code is emitted while parsing and every jump target is filled by backpatching, with no label-resolution pass.
3. Short-circuit boolean evaluation: `&&`, `||` and `!` compile to jumps, so the right operand is skipped once the result is known.
4. Clear error reporting: lexical errors, syntax errors and a `break` or `continue` outside a loop stop with their line number instead of crashing.

All four objectives are met. In addition, the project makes the backpatching process visible through a trace mode, and compares it with a two-pass implementation of the same language.

1.3 Scope

The source language is a C-like subset with integer variables only, no declarations, functions or arrays. Programs are sequences of statements; conditions use the relational operators <, <=, >, >=, == and !=. The output is three-address code, optionally printed as a quadruple table. Optimisation and target-code generation are outside the scope.

2. Literature Survey

The survey covers how compilers resolve forward jump targets and what each source contributes to this project.

2.1 Sources

Aho, Lam, Sethi and Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. (2006), ch. 6, sec. 6.6 (control flow) and sec. 6.7 (backpatching). This is the translation scheme the project implements: the synthesised attributes `truelist`, `falselist` and `nextlist`, the helpers `makelist`, `merge` and `backpatch`, and the empty marker non-terminals M and N. It is the baseline the generator is checked against.

Appel, Modern Compiler Implementation in C (1998), ch. 7. Conditionals are translated into intermediate trees whose jumps name symbolic labels; targets are resolved when the labels are placed, not by patching instruction numbers. This is the main alternative to backpatching and the basis of the two-pass baseline built for comparison (Section 4.6).

Cooper and Torczon, Engineering a Compiler, 2nd ed. (2011), ch. 7. Short-circuit evaluation of `&&` and `||` and the standard code shapes for `if`, `if-else` and loops, used to check that the generated jump sequences are the conventional ones.

Levine, flex & bison (2009). Practical Bison: `%union` semantic values, empty rules and mid-rule actions, and precedence declarations that resolve the dangling-else conflict. It directly informs the structure of `parser.y`.

Lattner and Adve, "LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation", CGO 2004. A production intermediate representation built from basic blocks: branch targets are blocks that can be created before their code exists, and each block's terminator is set once control flow is known. It shows how forward branches are handled outside the textbook setting.

Rodger and Finley, JFLAP: An Interactive Formal Languages and Automata Package (2006). A precedent for interactive, step-by-step visual tools in theory-of-computation teaching, which motivates the trace mode of this project.

2.2 Comparison of approaches

	Two-pass, symbolic labels	Backpatching (this project)	Basic-block IR (LLVM style)
Passes over the code	2: emit with labels, then resolve	1: targets filled during parsing	1 to build blocks; targets are block references
Memory for pending jumps	Label table plus all code held until resolution	Lists of unfilled jump indices	Block objects and the control-flow graph

	Two-pass, symbolic labels	Backpatching (this project)	Basic-block IR (LLVM style)
Fit with Bison	Easy, but needs a separate resolution pass	Natural: empty markers M and N capture the next instruction number	Heavy: needs an IR framework around the parser
Main source	Appel (1998)	Aho et al. (2006)	Lattner and Adve (2004)

Table 2.1 Three ways to resolve forward jumps

2.3 Gap

Backpatching is well-known textbook material, but its patch lists are invisible: a student sees the finished quadruples, not the moment each jump was created empty and later filled. Few student tools show the lists while code is generated. This project implements the textbook scheme in one pass and, as its extension, traces every list operation so that the process can be replayed step by step. It also measures the scheme against the two-pass alternative on the same programs.

3. System Design

3.1 Architecture

The generator is a single pass with no separate intermediate-representation pass. The Flex lexer (`lexer.l`) turns the source text into tokens and tracks line numbers. The Bison parser (`parser.y`) recognises the grammar, and the semantic action of each rule emits quadruples as the rule is reduced. The actions call four groups of helpers that live in the same file below the grammar: the quad table (`emit`, `newtemp`), the patch lists (`makelist`, `merge`, `backpatch`), the loop stack used by `break` and `continue`, and the symbol table. When parsing finishes, every jump has been filled and the code is printed as three-address code or as a quadruple table.

```
source --> lexer.l (Flex) --> parser.y (Bison) --> TAC, quads, trace
                                    |
                            semantic actions call
        +-------------+-------------+-------------+
        |             |             |             |
  symbol table   quad table    patch lists   loop stack
  symtab_insert  emit,         makelist,     loop_begin,
                 newtemp       merge,        loop_end,
                               backpatch     loop_jump
```

Figure 3.1 Architecture: one pass, all helpers inside parser.y

3.2 Source language

```
S -> id = E ;  |  if ( B ) S  |  if ( B ) S else S  |  while ( B ) S
   | do S while ( B ) ;  |  for ( id = E ; B ; id = E ) S
   | break ;  |  continue ;  |  { S ... }
B -> B || B  |  B && B  |  ! B  |  ( B )  |  E relop E  |  true  |  false
E -> E + E  |  E - E  |  E * E  |  E / E  |  - E  |  ( E )  |  id  |  num
```

Precedence, from lowest to highest, is `||`, `&&`, `!`, relational operators, `+ -`, `* /` and unary minus. `//` starts a comment that runs to the end of the line.

3.3 Translation scheme

Every non-terminal that produces jumps carries patch lists. For a condition B, `B.truelist` and `B.falselist` hold the numbers of the quadruples that must jump where control goes when B is true or false. For a statement S, `S.nextlist` holds the jumps that must go to the code following S. The global `nextinstr` is the number the next emitted quadruple will receive; numbering starts at 100.

Production	Semantic action
`program -> L`	backpatch(L.nextlist, nextinstr); emit(halt)
`L -> L1 M S`	backpatch(L1.nextlist, M.instr); L.nextlist = S.nextlist
`S -> id = E ;`	emit(id = E.addr); S.nextlist = empty
`S -> if ( B ) M S1`	backpatch(B.truelist, M.instr) S.nextlist = merge(B.falselist, S1.nextlist)

Production	Semantic action
`S -> if ( B ) M1 S1 else N M2 S2`	backpatch(B.truelist, M1.instr); backpatch(B.falselist, M2.instr) S.nextlist = merge(S1.nextlist, N.nextlist, S2.nextlist)
`S -> while M1 ( B ) W S1`	backpatch(S1.nextlist, M1.instr); backpatch(B.truelist, W.instr) emit(goto M1.instr); S.nextlist = merge(B.falselist, breaks) continues go to M1.instr
`S -> do W S1 while ( M B ) ;`	backpatch(S1.nextlist, M.instr); backpatch(B.truelist, W.instr) S.nextlist = merge(B.falselist, breaks); continues go to M.instr
`S -> for ( A1 ; M1 B ; M2 A2 N ) W S1`	backpatch(B.truelist, W.instr); backpatch(N.nextlist, M1.instr) backpatch(S1.nextlist, M2.instr); emit(goto M2.instr) S.nextlist = merge(B.falselist, breaks); continues go to M2.instr
`S -> break ;`	emit(goto _) and add it to the innermost loop's breaks
`S -> continue ;`	emit(goto _) and add it to the innermost loop's continues
`B -> B1 || M B2`	backpatch(B1.falselist, M.instr) B.truelist = merge(B1.truelist, B2.truelist); B.falselist = B2.falselist
`B -> B1 && M B2`	backpatch(B1.truelist, M.instr) B.truelist = B2.truelist; B.falselist = merge(B1.falselist, B2.falselist)
`B -> ! B1`	B.truelist = B1.falselist; B.falselist = B1.truelist
`B -> E1 relop E2`	B.truelist = makelist(nextinstr); B.falselist = makelist(nextinstr + 1) emit(if E1.addr relop E2.addr goto _); emit(goto _)
`B -> true | false`	truelist (or falselist) = makelist(nextinstr); emit(goto _)
`E -> E1 op E2 | - E1`	E.addr = newtemp(); emit(E.addr = E1.addr op E2.addr)
`M -> e`	M.instr = nextinstr
`N -> e`	N.nextlist = makelist(nextinstr); emit(goto _)
`W -> e`	W.instr = nextinstr; open a loop entry for break and continue

Table 3.1 Translation scheme (A is an assignment without the semicolon)

3.4 Marker non-terminals

M, N and W derive the empty string, so their reductions happen at precise points in the input, and their actions run at exactly those moments. M records `nextinstr` where the code of the following construct begins. N emits an unfilled `goto` that jumps over the else-branch, or, in a `for` loop, from the step back to the test. W is M that also pushes an entry on the loop stack, so that a `break` or `continue` in the body knows which loop it leaves.

The `for` loop shows why these markers are enough. Its parts appear in the source as init, test, step, body, but the step must run after the body. Because code is emitted in source order, the generated layout is init, test, step, `goto test`, body, `goto step`; the jumps between the pieces are exactly the lists patched in Table 3.1.

3.5 Data structures

Structure	Fields	Functions
Quadruple	kind, op, arg1, arg2, result, target (-1 until filled)	`emit, emit_goto`

Structure	Fields	Functions
Quad table	growable array; nextinstr starts at 100	`print_tac, print_quads`
Patch list	array of quadruple numbers; NULL is the empty list	`makelist, merge, backpatch`
Loop stack	break and continue jumps of each open loop, linked to the enclosing loop	`loop_begin, loop_end, loop_jump`
Symbol table	name and first-seen line (type is always int)	`symtab_insert`
Temporary generator	returns t1, t2, ...	`newtemp`

Table 3.2 Data structures in parser.y

The Bison `%union` holds `addr` (the name or temporary holding the value of E), `b` (the truelist and falselist of B), `nextlist` (for S, L and N) and `instr` (for M and W). `backpatch` refuses to fill a quadruple that is not an unfilled jump and stops with an internal error, which catches a wrong semantic action immediately instead of producing wrong code.

3.6 Conflicts and the dangling else

The grammar is ambiguous for a nested `if` with one `else`. The declarations `%nonassoc LOWER_THAN_ELSE` and `%nonassoc ELSE` give shifting `else` higher precedence than reducing the else-less `if`, so an `else` binds to the nearest `if`. In the textbook, N is placed before `else`; in Bison that produces a reduce/reduce conflict between N and the else-less `if`, which precedence cannot resolve. Moving N after `else` turns the choice into a shift/reduce one that precedence settles, and the emitted `goto` still lands directly after the then-branch because nothing is emitted while `else` is shifted. `%expect 0` makes the build fail on any conflict; both grammars build with none.

4. Implementation

The whole generator consists of three source files: `src/lexer.l` (Flex), `src/parser.y` (Bison grammar with C semantic actions and helpers) and `src/twopass.y` (the Bison baseline). No other language is used in the compiler itself; a Makefile builds both programs and a short shell script runs the tests.

4.1 Lexer

The lexer recognises the keywords `if`, `else`, `while`, `do`, `for`, `break`, `continue`, `true` and `false`, identifiers, integer literals, the relational operators, `&&`, `||`, `!` and single-character punctuation. Whitespace and `//` comments are skipped. Identifier, number and operator texts are copied into `yylval.str`. Any other character stops compilation with a message such as `line 2: unexpected character '$'`. The option `yylineno` supplies line numbers to all error messages. The same `lexer.l` serves both grammars, because they declare the same tokens.

4.2 Parser and semantic actions

Each rule of Table 3.1 is one Bison rule whose action is a direct transcription of the semantic action. For example, the `while` rule is:

```
| WHILE M '(' B ')' W S
                    { backpatch($7, $2);
                      backpatch($4.truelist, $6);
                      emit_goto($2);
                      $$ = merge($4.falselist, loop_end($2)); }
```

`emit` appends a quadruple to the table and returns its number; jumps start with the target UNFILLED, printed as `_`. `emit_goto` emits a backward jump whose target is already known, so it never enters a patch list. `makelist` creates a one-element list, `merge` concatenates two lists into a new one, and `backpatch` writes the target into every quadruple on a list. The empty list is NULL, so `merge` with an empty list costs nothing.

Short-circuit evaluation follows from the boolean rules. In `B1 && B2`, the true exits of B1 are patched to the start of B2, while its false exits join the false exits of the whole condition, so B2 is never evaluated when B1 is false. `!` generates no code: it only exchanges the two lists.

4.3 break and continue

A `break` or `continue` jumps to a place that is not known when it is parsed: the code after the loop, or the loop's test or step. The textbook scheme has no attribute for this, so the generator keeps a loop stack. The marker W pushes an entry when a loop body begins. `break` and `continue` emit an unfilled `goto` and add it to the innermost entry's `breaks` or `continues` list. When the loop rule is reduced, `loop_end` pops the entry, patches the continues to the loop's test (`while`), condition (`do-while`) or step (`for`), and returns the breaks, which join the loop's `nextlist`. A `break` or `continue` with no open loop stops with `line N: break outside a loop`.

4.4 Error handling

Bison is configured with `%define parse.error verbose`, and every token has a readable name, so a syntax error names what was expected, for example `line 3: syntax error, unexpected relational operator, expecting && or || or ')'` for `if (a < b < c)`. Lexical errors and misplaced `break` or `continue` are reported in the same `line N:` form. All errors stop compilation with a non-zero exit status and print no code.

4.5 Trace mode

With `--trace`, the generator prints each reduction that changes a patch list or emits a jump, followed by every operation it performs, and then the finished code after a blank line. Each list is shown by its contents, so a jump can be followed from the moment it is emitted as `goto _` to the `backpatch` line that fills it. With `--step`, the generator also waits for Enter after each operation, reading from the keyboard even when the program itself comes from standard input; this is meant for live walkthroughs. `--stats` reports the number of quadruples, the peak number of unfilled jumps and the number of patch-list entries allocated. The trace is implemented inside `parser.y` by a few lines in `emit`, `makelist`, `merge` and `backpatch`, and does not change the generated code.

4.6 Two-pass baseline

`twopass.y` is a second Bison grammar for the same language, implementing the label-based approach of Appel and of Aho et al. sec. 6.6. Pass 1 parses the program into a syntax tree. Pass 2 walks the tree, passing each statement the label of the code that follows it and each condition the labels to jump to when true and false; jumps name labels L1, L2, ... and each label records the quadruple it is placed before. Pass 3 replaces every label with its quadruple number. The code shapes match `parser.y`, so for any program both print the same quadruples. `--format=labels` prints the code after pass 2, before labels are resolved:

```
$ ./twopass --format=labels tests/03_if_else.src
      if a < b goto L3
      goto L4
L3:
      x = 1
      goto L2
L4:
      x = 2
L2:
      y = x
      halt
```

4.7 Build and test tooling

`make` runs Bison on each grammar, Flex on the lexer and gcc on the generated C, producing `icg` and `twopass`. Because `lexer.l` includes `parser.tab.h`, each grammar's header is generated into its own directory and the lexer is compiled once against each. `make test` runs `run_tests.sh`, which compiles every `tests/NN_name.src`, compares standard output and

standard error with `NN_name.expected`, and fails any test whose final code still contains `goto _`. A test may pass options through `NN_name.args`. Every test without options is also run through `twopass` and must produce exactly the same output.

5. Results and Testing

5.1 Test suite

The suite has 17 test programs. The 15 without options are run through both `icg` and `twopass`, giving 32 checks; all 32 pass. Each expected output was checked by hand against the translation scheme before it was recorded.

Test	What it covers	Result
`01_assign`	arithmetic, precedence, unary minus, temporaries	Pass
`02_if`	if without else	Pass
`03_if_else`	if-else and the jump over the else-branch	Pass
`04_while`	while loop, backward jump	Pass
`05_bool`	&&, || and ! with short-circuit jumps	Pass
`06_nested`	if-else nested in if-else nested in while	Pass
`07_do_while`	do-while loop	Pass
`08_syntax_error`	syntax error with expected tokens	Pass
`09_for`	for loop: init, test, step, body layout	Pass
`10_break_continue`	break and continue inside a for loop	Pass
`11_true_false`	constant conditions true and false	Pass
`12_break_outside`	break outside a loop is rejected	Pass
`13_lex_error`	unexpected character is rejected	Pass
`14_nested_loops`	break in an inner loop, continue in the outer	Pass
`15_dangling_else`	else binds to the nearest if	Pass
`16_trace`	--trace output for an if-else	Pass
`17_quad_format`	--format=quad table	Pass

Table 5.1 Test programs (tests 01-15 also pass through twopass)

In addition, during development 400 randomly generated programs with nested statements, loops, `break`, `continue` and compound conditions were compiled by both programs. All 400 produced identical output from both, none left an unfilled jump, and `--trace` never changed the generated code.

5.2 Short-circuit conditions

```
if (a < b && c > d || !(e == f)) x = 1;

100: if a < b goto 102
101: goto 104
102: if c > d goto 106
103: goto 104
104: if e == f goto 107
105: goto 106
106: x = 1
107: halt
```

If `a < b` is false (101), control skips `c > d` and goes straight to the right operand of `||` at 104. If both `a < b` and `c > d` hold, control reaches the assignment at 106 without testing `e == f`. Because of `!`, a true `e == f` makes the whole condition false (104 jumps to 107) and a false one makes it true (105 jumps to 106).

5.3 Loops with break and continue

```
for (i = 0; i < n; i = i + 1) {
    if (i == k) continue;
    if (s > 100) break;
    s = s + i;
}
t = s;

100: i = 0               110: goto 112
101: if i < n goto 106   111: goto 115
102: goto 115            112: t2 = s + i
103: t1 = i + 1          113: s = t2
104: i = t1              114: goto 103
105: goto 101            115: t = s
106: if i == k goto 108  116: halt
107: goto 109
108: goto 103
109: if s > 100 goto 111
```

The step `i = i + 1` (103-105) is emitted before the body because it appears first in the source; 105 returns to the test and 114 returns from the body to the step. The `continue` (108) was emitted as `goto _` and patched to the step, 103, when the loop closed; the `break` (111) was patched to 115, the statement after the loop, together with the loop's false exit 102.

5.4 Trace of an if-else

```
$ ./icg --trace tests/03_if_else.src
B -> E1 relop E2
    makelist   {100}
    makelist   {101}
    emit       100: if a < b goto _
    emit       101: goto _
    emit       102: x = 1
N -> e
    makelist   {103}
    emit       103: goto _
    emit       104: x = 2
S -> if ( B ) M1 S1 else N M2 S2
    backpatch  {100} := 102
    backpatch  {101} := 104
    emit       105: y = x
L -> L1 M S
    backpatch  {103} := 105
program -> L
    emit       106: halt
```

The trace shows the life of each jump. Quads 100 and 101 are created empty by the condition and filled when the whole `if-else` is reduced. Quad 103, the jump over the else-branch, stays empty longer: its target is the next statement, which is known only when `L -> L1 M S` is reduced after `y = x` has been emitted.

5.5 Comparison with the two-pass baseline

Table 5.2 compares the two programs using their `--stats` output. Both always generate the same number of quadruples. The backpatching generator holds only the jumps still waiting for a target; the peak is the largest number of unfilled jumps at any moment. The two-pass generator must hold the entire syntax tree until parsing ends, and then the label table and all the code until labels are resolved.

Program	Quads	Peak unfilled jumps	Patch-list entries	Tree nodes	Labels
`01_assign`	9	0	0	17	3
`02_if`	6	2	2	11	3
`03_if_else`	7	3	3	11	4
`04_while`	8	2	2	14	5
`05_bool`	8	5	10	15	4
`06_nested`	16	7	10	26	9
`07_do_while`	8	2	2	16	5
`09_for`	11	3	3	17	5
`10_break_continue`	17	8	11	29	9
`11_true_false`	9	3	4	17	8
`14_nested_loops`	18	7	12	32	11
`15_dangling_else`	8	5	7	12	4

Table 5.2 icg (columns 3-4) vs. twopass (columns 5-6) on the code-generating tests

On these programs at most 8 jumps are ever pending at once, at most two thirds of the number of quadruples and always well below the number of tree nodes, which grows with every expression and statement. The two-pass approach is simpler to reason about, because every statement receives its next label explicitly, but it needs the whole program in memory and an extra pass over the code. Backpatching delivers the same code in one pass, at the cost of the subtler marker non-terminals and patch lists that the trace mode makes visible.

6. Conclusion and Future Work

The project delivers a one-pass intermediate code generator, written entirely in Flex and Bison, that translates assignments, arithmetic, short-circuit conditions and all the usual control-flow statements of C, including `break` and `continue`, into three-address code by backpatching. The grammar has no conflicts, every jump is filled, and the output agrees exactly with an independent two-pass implementation on every test and on 400 random programs. The trace and step modes make the patch lists, normally invisible, observable at every step, which serves the teaching gap identified in the survey.

Possible extensions are:

- `switch` statements, whose case jumps fit naturally into patch lists.
- Arrays and procedure calls, which add address computation and parameter passing to the three-address code.
- Declarations and type checking in the symbol table, so that mixed types and undeclared variables are reported.
- Simple optimisations on the output, such as removing jumps to the next instruction and jumps to jumps, which the textbook code shapes produce (for example quad 105 in Section 5.2 jumps to 106).
- A graphical replay of the trace for classroom use, driven by the existing `--trace` output.

References

[1] A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman. Compilers: Principles, Techniques, and Tools, 2nd ed. Addison-Wesley, 2006. Ch. 6, sec. 6.6-6.7.

[2] A. W. Appel. Modern Compiler Implementation in C. Cambridge University Press, 1998. Ch. 7.

[3] K. D. Cooper, L. Torczon. Engineering a Compiler, 2nd ed. Morgan Kaufmann, 2011. Ch. 7.

[4] J. Levine. flex & bison. O'Reilly Media, 2009.

[5] C. Lattner, V. Adve. "LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation." Proc. International Symposium on Code Generation and Optimization (CGO), 2004, pp. 75-86.

[6] S. H. Rodger, T. W. Finley. JFLAP: An Interactive Formal Languages and Automata Package. Jones and Bartlett, 2006.

Appendix A. Building and running

Requirements: gcc, Flex and Bison (on Windows, `win_flex` and `win_bison` with `mingw32-make` from Git Bash). The source code is at github.com/Prithvi8706/ICG.

```
make                                    # builds ./icg and ./twopass
make test                               # 32 checks
./icg tests/04_while.src                # three-address code
./icg --format=quad tests/04_while.src # quadruple table
./icg --trace tests/03_if_else.src     # every list operation
./icg --step tests/03_if_else.src      # same, one step per Enter
./icg --stats tests/06_nested.src      # code size, pending jumps
./twopass --format=labels tests/04_while.src
./icg < program.src                     # read standard input
```